LAS CIEGAS HORMIGAS ELÉCTRICAS



Ingeniería Aumentada con Sistemas Inteligentes

Volumen III

Del método a la Metodología



Javier G. Grandez

v1.0 -2026

Tabla de contenidos

.1. Metodología IASI 5
Del método a la metodología 7
Cómo leer este volumen 9

I. Fundamentos de IASI 11
I.1. Qué es IASI 15
I.2. Principios metodológicos 17
I.3. Autoridad y fuentes de verdad 19
I.4. Artefactos y trazabilidad 21

II. La metodología 23
II.1. Conceptos y definiciones 27
II.2. Contenido y metadatos 35
II.3. Naturaleza de los artefactos 39
II.4. Autoridad y procedencia 43
II.5. Artefacto fuente IASI 47
II.6. De la fuente a la materialización 51
II.7. Inputs 55
II.8. Definitions 63
II.9. Rules y configuración 65
II.10. Specifications 67
II.11. Tasks 69
II.12. Validations 71
II.13. Estados y aceptación 73
II.14. Cambio e iteración 75

III. Estructura de un proyecto IASI 77
III.1. La raíz del proyecto 81
III.2. Estructura de inputs 83
III.3. Artefactos de ingeniería 85
III.4. Estado local de IASI 87
III.5. Directorios derivados de adapters 89
III.6. Outputs y releases 91

IV. Workflows 93
IV.1. Modelo de workflow 97
IV.2. De inputs a definitions 99
IV.3. Planificación y ejecución 101
IV.4. Validación y aceptación 103
IV.5. Workflow de cambio 105

V. Commands 107
- V.1. Modelo de command 111
- V.2. validate 113
- V.3. Commands de task 115
- V.4. archive 117
- V.5. adapter 119

VI. Adapters e integración 121
- VI.1. Por qué existen los adapters 125
- VI.2. Canónico y derivado 127
- VI.3. Adapters de agente 129
- VI.4. Agente, modelo y runtime 131
- VI.5. Capacidades y degradación 133

VII. Mecanismos transversales 135
- VII.1. Rules 139
- VII.2. Skills 141
- VII.3. Templates 143
- VII.4. Configuration 145
- VII.5. Automations 147
- VII.6. Ámbito y precedencia 149
- VII.7. Contenido derivado y contenido de autor 151

VIII Saco 153
- Referencias 157
- Licencias 159

.1. Metodología IASI

Volumen III · Ingeniería Aumentada con Sistemas Inteligentes



Del método a la metodología

IASI nació de una pregunta práctica: **¿cómo hacemos ingeniería cuando un Sistema Inteligente participa en el propio trabajo de ingeniería?**

Responderla exige algo más que aprender a utilizar un modelo, un agente o una interfaz.

Necesitamos saber qué información tiene autoridad, cómo se convierte en conocimiento operativo, cómo se define y representa el trabajo mediante tasks, cómo se ejecuta, cómo se valida, qué estado puede aceptarse y qué ocurre cuando cambian las fuentes de las que dependía una decisión.

Este volumen describe esa metodología.

No presenta una receta ligada a una herramienta concreta. Define un sistema de artefactos, responsabilidades, tasks, ciclos de vida, workflows, commands y adapters que permite trabajar sobre agentes y modelos diferentes conservando una semántica común.

La idea que atraviesa todo el volumen puede resumirse así:

```
metodología  → qué debe ocurrir
task         → qué trabajo concreto debe realizarse
estructura   → dónde vive
workflow     → cómo se organiza y recorre el trabajo
command      → cómo se solicita una operación
adapter      → cómo se ejecuta en un entorno concreto
```

La herramienta `iasi` materializa y automatiza partes de ese sistema.

Pero la herramienta no es la metodología.

Y el agente tampoco.

Cómo leer este volumen

Este volumen tiene dos naturalezas.

Por una parte, desarrolla una metodología. Conviene leer sus primeras partes en orden porque cada concepto prepara el siguiente: autoridad, artefactos, inputs, definitions, specifications, tasks, validations y estados forman un sistema.

Por otra, es también un documento de referencia. Las partes dedicadas a estructura, workflows, commands y adapters están pensadas para volver a ellas durante el trabajo cotidiano.

La terminología se mantiene deliberadamente estable. Cuando IASI utiliza palabras como `input`, `definition`, `specification`, `task`, `validation`, `state`, `workflow`, `command` o `adapter`, no las utiliza como sinónimos informales. Cada una representa una función distinta dentro del método.

El objetivo no es memorizar una jerarquía de carpetas o una colección de órdenes.

El objetivo es comprender **qué información posee autoridad, qué puede derivarse de ella, quién puede cambiarla, cómo se comprueba un resultado y cómo se conserva la historia de esas decisiones**.

Parte I.

Fundamentos de IASI

Una metodología no comienza por sus comandos.

Comienza por las decisiones que determinan **qué consideramos conocimiento, qué consideramos evidencia, quién tiene autoridad para decidir y cómo sabemos que un resultado puede aceptarse**.

IASI se apoya en un conjunto pequeño de principios que permanecen estables aunque cambien los modelos, los agentes, los lenguajes o las herramientas utilizadas para ejecutar el trabajo.

Esta parte establece esos principios antes de describir el ciclo metodológico.

I.1. Qué es IASI

IASI es una metodología de **Ingeniería Aumentada con Sistemas Inteligentes**.

Su propósito es permitir que personas y Sistemas Inteligentes participen en un mismo proceso de ingeniería sin convertir una conversación, un prompt o el comportamiento particular de una herramienta en la fuente de verdad del proyecto.

IASI define:

- artefactos con semántica explícita;
- relaciones entre esos artefactos;
- reglas de autoridad y precedencia;
- ciclos de vida y estados;
- mecanismos de validación;
- tasks que representan las tareas concretas que deben realizarse;
- workflows que organizan y recorren el proceso;
- commands que solicitan operaciones canónicas;
- adapters que proyectan esas operaciones sobre entornos concretos.

Una Task representa una unidad concreta de trabajo que debe realizarse. Puede ser ejecutada por una persona, un Sistema Inteligente, una herramienta o una combinación de ellos, y puede formar parte de un workflow más amplio.

IASI **no es un modelo de lenguaje**.

Tampoco es un agente central que gobierna el proyecto.

Un agente puede analizar, proponer, transformar, ejecutar o validar trabajo. Pero su autoridad no procede de ser «el agente de IASI». La autoridad permanece en las fuentes, los artefactos canónicos, las decisiones aprobadas y las responsabilidades humanas.

```
fuentes de autoridad
        ↓
artefactos de ingeniería
        ↓
tasks
        ↓
workflows y commands
        ↓
agentes · herramientas · personas
```

Esto permite utilizar IASI sobre entornos diferentes sin redefinir la metodología para cada uno.

La implementación `iasi` puede automatizar operaciones, comprobar estructura, materializar adapters o gestionar estados. Pero **la herramienta implementa la metodología; no la sustituye**.

I.2. Principios metodológicos

IASI parte de una idea sencilla: una conversación es excelente para pensar, pero es un lugar frágil para conservar el conocimiento de ingeniería.

Una decisión importante puede escribirse en un chat o en un documento con un coste prácticamente idéntico. En el segundo caso, además, puede quedar versionada, revisada, trazada, reutilizada y procesada por herramientas.

Por eso IASI intenta que el conocimiento relevante **termine materializado en artefactos persistentes**.

De este principio se derivan otros.

I.2.1. El prompt no es la fuente de verdad

El prompt puede iniciar una acción y aportar contexto transitorio. No debe convertirse en el único lugar donde viven requisitos, decisiones, restricciones o criterios de aceptación.

I.2.2. La evidencia y la interpretación se separan

Lo recibido no se reescribe para hacerlo más cómodo.

Especialmente, `inputs/externals/` conserva exactamente aquello que llegó desde fuera. La normalización y la interpretación producen nuevos artefactos.

I.2.3. No se actúa más allá de lo conocido

La falta de información no obliga siempre a detener todo el trabajo. Limita el alcance de aquello que puede hacerse legítimamente.

IASI puede avanzar hasta donde permitan la información disponible, las restricciones y el alcance explícito. Lo que no puede hacer es completar silenciosamente los huecos mediante invención.

I.2.4. Lo derivado no sustituye a su fuente

Código, documentos, configuraciones, adapters y otros resultados pueden ser materializados a partir de artefactos anteriores.

Cuando eso ocurre debe conservarse la relación entre fuente y derivado.

I.2.5. La validación es parte del proceso

La generación de un resultado no demuestra que sea correcto.

IASI separa ejecución y validación y conserva evidencia suficiente para justificar el estado alcanzado.

I.2.6. Las herramientas son sustituibles

Un command de IASI debe conservar su significado aunque cambie el agente que lo ejecuta.

Los adapters absorben las particularidades del entorno; la metodología permanece.

I.2.7. La responsabilidad no se delega al Sistema Inteligente

El Sistema Inteligente puede aumentar la capacidad de quien trabaja con él.

La decisión profesional, la autoridad y la responsabilidad siguen perteneciendo a las personas y roles que las tienen asignadas.

I.3. Autoridad y fuentes de verdad

Uno de los problemas que aparecen al incorporar Sistemas Inteligentes a la ingeniería es confundir **capacidad de actuación** con **autoridad**.

Un agente puede ser capaz de modificar cien archivos. Eso no significa que pueda decidir legítimamente qué debe contener el sistema.

IASI distribuye la autoridad entre artefactos con responsabilidades diferentes:

```
inputs              → de dónde procede la información
definitions         → qué hemos comprendido
rules               → qué normas deben respetarse
configuration       → con qué parámetros trabajamos
specifications      → qué resultado se pretende obtener
tasks               → qué trabajo concreto debe ejecutarse
validations         → qué evidencia demuestra el resultado
states              → cuál es el estado materializado del proceso
```

Ninguna de estas piezas debe interpretarse aisladamente.

Una `specification` no puede contradecir una `rule` aplicable. Una `task` no debe inventar alcance que no exista en su specification. Un estado `done` carece de valor si no puede relacionarse con el trabajo y las validaciones que lo justifican.

La precedencia tampoco se reduce a «lo último gana».

Cada clase de artefacto posee sus propias reglas. En el caso de las rules, por ejemplo, una regla más específica puede prevalecer sobre una genérica cuando ambas regulan el mismo aspecto. En estructuras heredables, el ámbito más próximo puede refinar el superior.

La idea común es:

> **La autoridad debe ser explícita y resoluble.**

Cuando dos fuentes incompatibles reclaman autoridad sobre el mismo aspecto, el sistema no debería elegir silenciosamente la interpretación más conveniente. Debe detectar el conflicto y resolverlo mediante las reglas correspondientes o mediante una decisión humana.

I.4. Artefactos y trazabilidad

IASI trabaja con artefactos porque los artefactos permiten conservar el razonamiento de ingeniería fuera de la memoria de una conversación.

No todos los artefactos tienen la misma naturaleza.

Podemos distinguir, al menos, tres grandes clases:

```
fuentes
  inputs

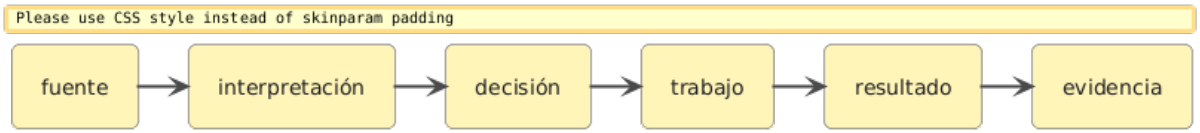
artefactos canónicos de ingeniería
  definitions
  rules
  configuration
  specifications
  tasks
  validations
  states

artefactos derivados
  código
  documentación generada
  configuraciones materializadas
  adapters
  paquetes
  publicaciones
```

La distinción importa especialmente cuando una parte del sistema puede regenerarse.

Si un documento se deriva de unas definitions, modificar manualmente el resultado sin conservar esa decisión rompe la trazabilidad. La siguiente materialización podría perderla.

Por eso IASI intenta conservar una cadena de procedencia:



La trazabilidad no pretende registrar cada token producido por un modelo.

Pretende poder responder preguntas de ingeniería:

- ¿por qué existe este comportamiento?;
- ¿qué necesidad lo originó?;
- ¿qué specification lo exige?;
- ¿qué task lo materializó?;
- ¿qué validación lo comprobó?;
- ¿qué cambio posterior podría afectarlo?

Cuando estas relaciones pueden reconstruirse, el proyecto deja de depender de recordar «qué hablamos aquel día».

Parte II.

La metodología

La metodología IASI organiza cómo se representa el conocimiento de ingeniería y cómo ese conocimiento se transforma en trabajo y resultados aceptados.

Esta parte se divide en dos planos relacionados.

El primero define el **modelo semántico**: qué es un artefacto, qué diferencia existe entre contenido y metadatos, cómo se representan autoridad y procedencia, y cómo se relacionan las fuentes con sus materializaciones.

```
conceptos
   ↓
contenido + metadatos
   ↓
naturaleza de los artefactos
   ↓
autoridad + procedencia
   ↓
artefacto fuente IASI
   ↓
materialización
```

El segundo describe el **ciclo metodológico de trabajo**, desde la información de entrada hasta la aceptación del resultado:

```
inputs
   ↓
definitions
   ↓
rules + configuration
   ↓
specifications
   ↓
tasks
   ↓
validations
   ↓
acceptance + states
```

No son dos metodologías distintas.

El modelo semántico establece **qué significa** la información y cómo debe tratarse. El ciclo metodológico establece **cómo progresa** esa información dentro del trabajo de ingeniería.

Tampoco se trata de una cascada irreversible. Una task puede descubrir nueva información; una validation puede revelar que una definition era incompleta; un nuevo input puede afectar a specifications ya existentes. IASI conserva las relaciones necesarias para volver atrás sin perder la historia ni confundir autoridad, procedencia y estado.

Los capítulos de esta parte desarrollan primero la semántica y después el ciclo operativo construido sobre ella.

II.1. Conceptos y definiciones

Este capítulo establece el vocabulario de la metodología IASI.

Las definiciones no pretenden crear simples etiquetas para ficheros. Su objetivo es fijar las propiedades semánticas que permiten razonar sobre la información: qué dice, de dónde procede, quién tiene autoridad sobre ella, cómo ha sido obtenida, si puede modificarse, regenerarse o descartarse, y cuál es su papel dentro de un proceso IASI.

Un principio fundamental es que estas propiedades **no constituyen necesariamente un único tipo**. IASI debe distinguir dimensiones independientes en lugar de comprimirlas en una clasificación plana.

II.1.1. Resumen de conceptos

	Concepto	Resumen
1	Adopción (`Adoption`)	Aceptación explícita de información dentro de una frontera de autoridad.
2	Artefacto (`Artifact`)	Unidad identificable de información, conocimiento, configuración, evidencia o resultado que participa en IASI.
3	Artefacto fuente (`Source Artifact`)	Artefacto que actúa como entrada de un proceso de interpretación, razonamiento, transformación o materialización.
5	Autoría asumida (`Authored`)	Contenido deliberadamente afirmado, aceptado o mantenido dentro de una frontera de autoridad IASI.
4	Autoridad (`Authority`)	Capacidad reconocida para establecer el significado o estado aceptado de un artefacto dentro de un ámbito.
6	Canónico (`Canonical`)	Artefacto o representación designado como referencia dentro de un ámbito concreto.
7	Contenido (`Content`)	Lo que un artefacto dice o contiene.
8	Derivado (`Derived`)	Artefacto producido a partir de otros mediante una transformación definida.
9	Desechable (`Disposable`)	Artefacto que puede eliminarse sin destruir información autoritativa.
10	Efímero (`Ephemeral`)	Artefacto cuya vida útil está concebida como temporal.
11	Externo (`External`)	Contenido cuya fuente autoritativa se encuentra fuera de la frontera de autoridad IASI considerada.
12	Inferido (`Inferred`)	Información obtenida mediante razonamiento a partir de otras evidencias y no mediante afirmación explícita de una fuente autoritativa.
13	Materialización (`Materialization`)	Proceso de producir artefactos concretos a partir de fuentes, metadatos, reglas y transformaciones.
14	Metadatos (`Metadata`)	Información que describe qué es un artefacto y cómo debe ser interpretado o tratado.
15	Procedencia (`Provenance`)	Rastro de dónde procede la información y de las transformaciones o decisiones que la han llevado a su estado actual.

	Concepto	Resumen
16	Regenerable (`Regenerable`)	Artefacto que puede reconstruirse a partir de entradas autoritativas y reglas de transformación conocidas.

II.1.2. Adopción (`Adoption`)

La **adopción** es el acto explícito mediante el cual un actor con autoridad acepta información dentro de una frontera de autoridad.

Es especialmente importante para información inferida o externa.

Por ejemplo, un Sistema Inteligente puede inferir una restricción a partir de varios requisitos. Esa conclusión conserva su origen inferido. Si el ingeniero responsable la revisa y la acepta como parte de la especificación, pasa a ser autoritativa dentro del ámbito correspondiente.

La adopción añade autoridad, pero **no debe borrar la procedencia**.

II.1.3. Artefacto (`Artifact`)

Un **artefacto** es una unidad identificable de información, conocimiento, especificación, configuración, evidencia o resultado que participa en un proceso IASI.

Puede estar representado por un fichero, un fragmento documental, un registro, un recurso externo, un activo generado u otra unidad direccionable.

La representación física no define por sí sola la naturaleza metodológica del artefacto.

II.1.4. Artefacto fuente (`Source Artifact`)

Un **artefacto fuente** es un artefacto que participa como entrada de un proceso de interpretación, razonamiento, transformación o materialización.

No tiene por qué ser `Authored`. Puede ser externo, inferido y posteriormente adoptado, o cualquier otra fuente aceptada por el proceso.

Fuente describe un papel en un proceso, no una autoridad.

II.1.5. Autoridad (`Authority`)

La **autoridad** identifica quién o qué está legitimado para establecer el significado o estado aceptado de un artefacto dentro de un ámbito determinado.

No debe confundirse con:

- autoría física;
- posesión del fichero;
- almacenamiento local;
- modificación más reciente;
- generación;
- inferencia.

Un artefacto puede estar almacenado localmente y seguir siendo autoritativamente externo.

Una conclusión inferida puede ser útil sin poseer autoridad hasta que sea adoptada.

II.1.6. Autoría asumida (`Authored`)

`Authored` describe contenido que ha sido **deliberadamente afirmado, aceptado o mantenido** dentro de una frontera de autoridad IASI.

No significa simplemente «escrito por una persona».

Un humano puede crear contenido con una participación extensa de un Sistema Inteligente y seguir asumiendo su autoría metodológica. Del mismo modo, un texto tecleado enteramente por una persona puede no ser `Authored` si es una nota temporal, una propuesta todavía no aceptada o una copia de una fuente externa.

`Authored` implica, por tanto, **adopción consciente y responsabilidad**, no autoría mecánica de las palabras.

II.1.7. Canónico (`Canonical`)

Un artefacto o representación es **canónico** cuando ha sido designado como referencia para un ámbito concreto.

Canónico no significa universalmente autoritativo.

Una especificación IASI puede ser canónica para una implementación y, al mismo tiempo, depender de una norma externa que sigue siendo la autoridad respecto a determinadas reglas.

II.1.8. Contenido (`Content`)

El **contenido** es lo que un artefacto dice o contiene.

Responde a la pregunta:

> ¿Qué dice esto?

Son contenido, por ejemplo, requisitos, decisiones de diseño, especificaciones, explicaciones, código fuente, diagramas, observaciones y conclusiones.

El contenido es distinto de las reglas que determinan cómo debe interpretarse o tratarse ese contenido.

II.1.9. Derivado (`Derived`)

`Derived` describe un artefacto producido a partir de otros artefactos mediante una transformación definida.

Son ejemplos habituales:

- HTML renderizado;
- documentación generada;
- esquemas compilados;
- informes;
- paquetes de release;
- diagramas generados;
- conversiones de formato;
- índices calculados.

Un derivado es normalmente **regenerable**. Su significado autoritativo procede de sus entradas y de las reglas de transformación, no de modificaciones manuales sobre el resultado generado.

Por ello, una modificación directa de un artefacto derivado suele ser inválida o descartable: la corrección debe realizarse aguas arriba.

II.1.10. Desechable (`Disposable`)

Un artefacto es **desechable** cuando puede eliminarse sin destruir información autoritativa.

Los artefactos derivados y efímeros son frecuentemente desechables.

Desechable no significa inútil. Un artefacto puede ser imprescindible durante una ejecución y, aun así, no constituir una fuente de verdad que deba conservarse.

II.1.11. Efímero (`Ephemeral`)

`Ephemeral` describe un artefacto cuya vida útil está concebida intencionadamente como temporal.

Ejemplos:

- directorios de trabajo;
- cachés;
- ficheros intermedios;
- resultados de cálculo temporales;
- prompts transitorios;
- exportaciones temporales;
- áreas de staging.

`Ephemeral` expresa principalmente una **propiedad de ciclo de vida**.

Un artefacto puede ser a la vez derivado y efímero. También puede ser `Authored` y efímero, como una nota de trabajo todavía no incorporada al cuerpo durable de conocimiento.

II.1.12. Externo (`External`)

`External` describe contenido cuya fuente autoritativa se encuentra fuera de la frontera de autoridad IASI considerada.

Pueden ser externos:

- normas;
- legislación;
- documentación de fabricantes;
- APIs mantenidas por terceros;
- investigación publicada;
- requisitos suministrados por un cliente;
- repositorios propiedad de otra organización.

IASI puede referenciar, copiar, almacenar en caché, anotar o derivar información de un artefacto externo.

Una copia local **no transfiere silenciosamente la autoridad** al proyecto IASI.

II.1.13. Inferido (`Inferred`)

`Inferred` describe información obtenida mediante razonamiento a partir de otras evidencias, en lugar de haber sido afirmada explícitamente por una fuente autoritativa.

La inferencia puede ser producida por:

- una persona;
- un Sistema Inteligente;
- un motor de reglas;
- un analizador estático;
- cualquier otro proceso de razonamiento.

`Inferred` describe **cómo se obtuvo una afirmación**. Por sí solo, no concede autoridad.

Una inferencia debería conservar suficiente procedencia para responder, cuando sea relevante, a preguntas como:

- ¿a partir de qué evidencias se obtuvo?;
- ¿qué proceso o agente realizó la inferencia?;
- ¿bajo qué supuestos?;
- ¿con qué grado de confianza?;
- ¿ha sido revisada?;
- ¿ha sido adoptada?

Una afirmación inferida puede convertirse posteriormente en información autoritativa mediante adopción sin dejar de conservar su origen inferido.

II.1.14. Materialización (`Materialization`)

La **materialización** es el proceso de producir artefactos concretos a partir de artefactos fuente, metadatos, reglas y transformaciones.

Los resultados materializados son normalmente derivados.

Ejemplos:

- HTML;
- PDF;
- presentaciones;
- configuraciones generadas;
- paquetes de distribución;
- árboles de publicación.

Materializar no significa crear una nueva fuente de verdad. Significa producir una representación utilizable a partir de fuentes y reglas conocidas.

II.1.15. Metadatos (`Metadata`)

Los **metadatos** describen el significado metodológico de un artefacto y cómo debe tratarse.

Responden a preguntas como:

¿Qué es esto?
¿De dónde procede?
¿Quién tiene autoridad sobre ello?
¿Puede modificarse?
¿Está vigente?
¿De qué depende?
¿Puede regenerarse?
¿Es temporal?

Los metadatos forman parte de la semántica IASI aunque físicamente se representen mediante front matter, un fichero lateral, una base de datos, un grafo u otro mecanismo.

Front matter es una representación. Metadata es el concepto.

II.1.16. Procedencia (`Provenance`)

La **procedencia** es el rastro de dónde proviene una información y cómo ha llegado a su forma actual.

Puede incluir:

- artefactos fuente;
- referencias externas;
- autores o agentes;
- pasos de inferencia;
- transformaciones;
- versiones;
- marcas temporales;
- revisiones;
- eventos de adopción.

Responde a la pregunta:

¿Cómo ha llegado esta información a ser lo que es ahora?

La procedencia debe sobrevivir a las transformaciones y a la adopción. Ganar autoridad no debe borrar la historia de la información.

II.1.17. Regenerable (`Regenerable`)

Un artefacto es **regenerable** cuando puede volver a crearse a partir de entradas autoritativas y reglas de transformación definidas.

La regenerabilidad es un fuerte indicio de que ese artefacto no debería convertirse en una fuente de verdad independiente.

Si un resultado puede reconstruirse de manera fiable, la metodología debe favorecer la corrección de sus fuentes o reglas antes que la edición manual del resultado.

II.1.18. Distinciones fundamentales

Estos conceptos están relacionados, pero no son sinónimos.

En particular:

- `Authored`, `External` e `Inferred` hablan de aspectos de origen, adopción o autoridad;
- `Derived` habla de producción mediante transformación;
- `Ephemeral` habla de ciclo de vida;
- autoridad y procedencia son dimensiones distintas;
- fuente describe un papel dentro de un proceso;
- canónico describe una referencia dentro de un ámbito.

IASI no debería colapsar estas dimensiones en un único campo `type` salvo que el modelo demuestre que esa simplificación conserva toda la semántica necesaria.

II.2. Contenido y metadatos

IASI distingue entre **lo que dice un artefacto** y **lo que ese artefacto significa metodológicamente**.

Esta distinción es fundamental.

II.2.1. Dos preguntas diferentes

El contenido responde a:

> ¿Qué dice esto?

Los metadatos responden a:

> ¿Qué es esto y cómo debe tratarse?

Consideremos dos ficheros que contienen exactamente el mismo Markdown:

```
# Política de contraseñas

Las contraseñas deben contener al menos catorce caracteres.
```

Sintácticamente, ambos ficheros son idénticos.

Metodológicamente, pueden ser completamente distintos.

Uno puede ser un requisito de seguridad asumido internamente. Otro puede ser una cita de una norma externa. Un tercero puede haber sido generado a partir de una base de datos de políticas. Un cuarto puede ser una copia temporal de trabajo.

El contenido textual, por sí solo, no permite a IASI saber cómo debe tratar cada caso.

II.2.2. La sintaxis no es la semántica

El formato de fichero no determina su significado metodológico.

Un fichero `.md` no es automáticamente `Authored`.
Un PDF descargado no es automáticamente autoritativo.
Un fichero generado no adquiere autoridad por el mero hecho de existir en un repositorio.
Una copia local de un documento externo no transfiere su autoridad al proyecto local.

IASI necesita, por tanto, semántica más allá de la sintaxis.

II.2.3. Los metadatos como semántica metodológica

Los metadatos pueden expresar propiedades como:

- origen;
- autoridad;
- procedencia;
- ciclo de vida;
- estado;
- versión;
- vigencia;
- relaciones;
- dependencias;
- historial de transformaciones;
- estado de revisión;
- estado de adopción.

El esquema exacto puede evolucionar, pero la separación entre contenido y metadatos debe permanecer estable.

II.2.4. El front matter es un mecanismo de representación

El front matter es una forma física cómoda de asociar metadatos a contenido textual.

No es el modelo conceptual.

Por ejemplo:

```
+++
origin = "external"
authority = "external"
status = "current"
+++

# Contenido de ejemplo

...
```

Los conceptos metodológicos seguirían existiendo si los metadatos estuvieran almacenados en una base de datos, un fichero lateral, un grafo, una API o un contenedor binario.

IASI define primero la semántica y después su serialización.

II.2.5. Por qué importa para los Sistemas Inteligentes

Un Sistema Inteligente puede leer texto y generar texto plausible. Eso no basta para hacer ingeniería.

También debe poder distinguir:

- hechos de inferencias;
- autoridad de sugerencia;
- fuente de resultado generado;
- conocimiento durable de contexto temporal;
- restricción externa de decisión interna.

Sin metadatos, todo el texto tiende a aplanarse dentro de un mismo contexto indiferenciado.

Y con ello se pierde significado de ingeniería.

II.2.6. El mismo contenido puede requerir tratamientos distintos

Supongamos que la siguiente frase aparece en varios artefactos:

> El servicio conservará los logs de auditoría durante 365 días.

Puede ser:

1. **External**: copiada de una regulación.
2. **Authored**: adoptada como requisito interno.
3. **Inferred**: deducida por un Sistema Inteligente a partir de varias declaraciones de cumplimiento.
4. **Derived**: materializada dentro de una matriz de cumplimiento generada.
5. **Ephemeral**: presente en una nota temporal de análisis.

La frase es idéntica.

Su significado de ingeniería no lo es.

II.2.7. El contenido no debe ocultar metodología

Un fallo recurrente consiste en codificar estado metodológico de forma informal dentro de la prosa:

```
NOTA: esto viene del cliente y probablemente no debería modificarse.
```

La información es valiosa, pero no es fiable como dato procesable por una máquina.

IASI debe preferir metadatos explícitos para las propiedades que afecten al procesamiento, la autoridad, la validación o el ciclo de vida.

La explicación legible por personas puede complementar a los metadatos, pero no sustituirlos.

II.2.8. Principio

El contenido transporta conocimiento. Los metadatos transportan semántica metodológica.

Ambos cooperan, pero siguen siendo planos distintos.

II.3. Naturaleza de los artefactos

Los artefactos IASI no pueden comprenderse adecuadamente mediante una clasificación plana y única.

Términos como `Authored`, `External`, `Inferred`, `Derived` y `Ephemeral` describen aspectos diferentes de un artefacto. Algunos se refieren a autoridad, otros a procedencia, producción o ciclo de vida.

Tratarlos como valores mutuamente excluyentes de un único campo `type` ocultaría combinaciones importantes.

II.3.1. Un modelo multidimensional

Un artefacto puede necesitar responder a varias preguntas independientes.

II.3.1.1. Origen

¿Cómo entró la información en el sistema?

Ejemplos posibles:

- authored;
- external;
- inferred.

II.3.1.2. Autoridad

¿Quién establece el significado aceptado?

Ejemplos posibles:

- interna;
- externa;
- ninguna;
- delegada.

II.3.1.3. Producción

¿Cómo se creó la representación actual?

Ejemplos posibles:

- creada directamente;
- derivada;
- materializada.

II.3.1.4. Ciclo de vida

¿Cuál es su persistencia prevista?

Ejemplos posibles:

- durable;
- efímera.

II.3.1.5. Estado

¿Cuál es su estado metodológico actual?

Ejemplos posibles:

- draft;
- reviewed;
- adopted;
- superseded;
- obsolete.

Estas dimensiones son ilustrativas. El esquema definitivo de metadatos debe emerger de la metodología, no al revés.

II.3.2. `Authored` es un acto de responsabilidad

`Authored` significa que un artefacto o afirmación ha sido deliberadamente afirmado o adoptado dentro de la frontera de autoridad correspondiente.

Esto permite una distinción importante:

```
generado por SI
      |
      v
revisado por ingeniero
      |
      v
adoptado
      |
      v
Authored dentro de IASI
```

El Sistema Inteligente puede haber producido las palabras, pero la autoridad de ingeniería procede de la adopción.

II.3.3. `External` habla de la frontera de autoridad

`External` no significa remoto.

Un artefacto externo puede estar físicamente copiado dentro del repositorio.

Sigue siendo externo si su autoridad permanece fuera del proyecto.

A la inversa, un artefacto creado internamente puede almacenarse en un servicio externo y seguir siendo autoritativo internamente.

Ubicación física y autoridad metodológica son dimensiones diferentes.

II.3.4. `Inferred` habla del origen epistémico

La información `Inferred` no ha sido afirmada explícitamente por la evidencia fuente.

Ha sido razonada a partir de esa evidencia.

Esta distinción es especialmente importante en ingeniería aumentada, porque los Sistemas Inteligentes producen continuamente afirmaciones útiles que pueden resultar plausibles, coherentes e incluso correctas sin ser autoritativas.

IASI debe preservar la diferencia entre:

```
afirmado
inferido
adoptado
```

No son estados intercambiables.

II.3.5. `Derived` habla de reproducibilidad

Un artefacto `Derived` existe porque se ha aplicado una transformación a unas entradas.

Su propiedad definitoria no es simplemente que una herramienta lo haya creado.

La pregunta relevante es:

> ¿Puede recrearse este artefacto a partir de fuentes y reglas conocidas?

Si la respuesta es sí, las modificaciones manuales sobre el derivado deberían considerarse normalmente inestables.

La corrección pertenece aguas arriba.

II.3.6. `Ephemeral` habla de temporalidad intencional

Los artefactos `Ephemeral` soportan el trabajo sin convertirse en parte del cuerpo durable de conocimiento.

Ejemplos:

- cachés;
- árboles temporales de renderizado;
- prompts de trabajo;
- cálculos intermedios;
- directorios de staging.

Efímero no significa irrelevante.

Un artefacto temporal puede ser esencial durante una ejecución. Su característica definitoria es que la metodología no depende de conservarlo como verdad durable.

II.3.7. Combinaciones útiles

El valor de un modelo multidimensional se hace visible en las combinaciones.

II.3.7.1. External + Durable

Una copia local de una regulación externa conservada para trazabilidad.

II.3.7.2. Inferred + Ephemeral

Una hipótesis temporal producida por un Sistema Inteligente durante un análisis.

II.3.7.3. Inferred + Adopted + Durable

Una conclusión originalmente inferida por un Sistema Inteligente, revisada por un ingeniero y aceptada dentro de la especificación del proyecto.

II.3.7.4. Derived + Durable

Un PDF de release conservado como artefacto de publicación aunque pueda regenerarse.

II.3.7.5. Derived + Ephemeral

Un árbol HTML temporal creado durante un build.

II.3.7.6. Authored + Ephemeral

Una nota humana de trabajo que todavía no ha sido adoptada dentro de la metodología durable.

II.3.8. Principio

IASI debe modelar **propiedades**, no forzar cada artefacto dentro de una única caja.

Un buen modelo semántico permite combinaciones y, al mismo tiempo, conserva las distinciones necesarias para autoridad, trazabilidad, regeneración, revisión y automatización.

II.4. Autoridad y procedencia

Autoridad y procedencia responden a preguntas diferentes.

Autoridad pregunta quién puede establecer el significado aceptado.

Procedencia pregunta de dónde viene la información y cómo ha llegado a su forma actual.

Ambas son necesarias.

II.4.1. Autoridad no es autoría

Una persona puede escribir un texto sin tener autoridad sobre su materia.

Una herramienta puede generar un texto que posteriormente sea aceptado por un ingeniero con autoridad.

Una norma externa copiada en local puede almacenarse dentro del proyecto sin convertirse en autoritativa internamente.

Por tanto, la autoridad no puede inferirse de quién tecleó los caracteres.

II.4.2. Fronteras de autoridad

IASI opera dentro de fronteras de autoridad explícitas o implícitas.

Ejemplos:

- un proyecto;
- una organización;
- un producto;
- un dominio regulatorio;
- un organismo externo de normalización;
- una especificación controlada por un cliente.

Un artefacto puede ser autoritativo en un ámbito y no serlo en otro.

Por ejemplo, una especificación interna de API puede ser canónica y autoritativa para la implementación de un servicio y, al mismo tiempo, estar subordinada a una norma de protocolo externa que mantiene la autoridad sobre determinadas reglas.

II.4.3. La procedencia debe sobrevivir a las transformaciones

Cuando la información atraviesa IASI, su historia no debe desaparecer.

Consideremos:

```
Regulación externa
        |
        v
Análisis interno
        |
        v
Inferencia del SI
        |
        v
Revisión del ingeniero
        |
        v
Requisito adoptado
        |
        v
Informe de cumplimiento generado
```

Cada etapa modifica el papel o el estado de la información.

El informe final no debería hacer invisible esa cadena.

II.4.4. Una inferencia necesita evidencia

Una afirmación inferida debe poder trazarse hasta sus evidencias.

Como mínimo, un modelo de inferencia debería poder representar:

- artefactos de entrada;
- agente o proceso de razonamiento;
- supuestos;
- contexto temporal o de versión;
- confianza, cuando sea relevante;
- estado de revisión;
- estado de adopción.

Esto es especialmente importante con Sistemas Inteligentes, porque un lenguaje fluido puede ocultar incertidumbre.

IASI debe tratar la procedencia como dato de ingeniería, no como atribución decorativa.

II.4.5. La adopción añade autoridad

La adopción es una transición.

Antes de la adopción:

```
afirmación inferida
authority = none o provisional
```

Después de una revisión autorizada:

```
misma afirmación
origin = inferred
authority = internal
state = adopted
```

La procedencia original debe mantenerse intacta.

IASI no debe reescribir la historia fingiendo que la información adoptada fue siempre directamente `Authored`.

II.4.6. La información externa puede adoptarse sin perder su origen

Una afirmación externa puede convertirse en un requisito interno.

Por ejemplo:

```
cláusula de norma externa
        |
        v
decisión interna de cumplimiento
        |
        v
requisito del proyecto
```

El requisito del proyecto es autoritativo internamente.

Su justificación sigue apuntando a la fuente externa.

Esta separación permite a IASI responder a dos preguntas distintas:

> ¿Por qué exigimos esto?

Y:

> ¿Quién es responsable actualmente de este requisito?

II.4.7. Conflictos de autoridad

Pueden aparecer conflictos cuando:

- un requisito interno contradice una restricción externa;
- dos fuentes externas discrepan;
- una conclusión inferida entra en conflicto con contenido `Authored`;
- una representación derivada parece inconsistente con su fuente.

IASI no debería resolver estos conflictos por fecha de fichero ni mediante precedencias arbitrarias.

La metodología debe utilizar información explícita sobre autoridad y relaciones.

II.4.8. Principio

La autoridad determina quién puede decidir. La procedencia conserva cómo hemos llegado hasta allí.

La trazabilidad de ingeniería necesita ambas.

II.5. Artefacto fuente IASI

IASI necesita una representación fuente capaz de transportar tanto contenido legible por personas como semántica metodológica.

La propuesta inicial es el artefacto `.iasi`.

II.5.1. La extensión `.iasi`

Un fichero `.iasi` es un artefacto fuente IASI.

No es simplemente Markdown con otra extensión.

Su significado procede de la metodología IASI y del contrato definido para el formato.

La representación inicial combina:

1. front matter TOML para los metadatos metodológicos;
2. contenido Markdown para el conocimiento legible por personas.

Conceptualmente:

```
+---------------------------+
| metadatos TOML            |
| "¿Qué es esto?"           |
| "¿Cómo debe tratarse?"    |
+---------------------------+
| contenido Markdown        |
| "¿Qué dice esto?"         |
+---------------------------+
```

II.5.2. Forma física inicial

Un ejemplo mínimo podría ser:

```
+++
title = "Requisitos de autenticación"
status = "draft"
+++

# Requisitos de autenticación

El servicio deberá...
```

Este capítulo no congela deliberadamente el esquema exacto de metadatos.

IASI debe definir primero el modelo conceptual.

La serialización viene después.

II.5.3. Por qué TOML

IASI utiliza TOML para configuración y metadatos estructurados mantenidos por personas, salvo que un caso de uso concreto requiera otra representación.

TOML proporciona una representación explícita y legible sin convertir YAML en parte del contrato central de IASI.

II.5.4. Por qué Markdown

Markdown proporciona un sustrato práctico y legible para prosa, listas, tablas, código, enlaces y escritura técnica estructurada.

Sin embargo, Markdown es la representación del contenido, no la metodología.

IASI debe conservar la libertad de definir semántica por encima de él.

II.5.5. Independencia respecto a las herramientas

Un fichero `.iasi` no debe definirse como:

- un documento Quarto;
- un documento Pandoc;
- un documento de VS Code;
- un prompt de Copilot;
- un documento knitr.

Esos sistemas pueden actuar como procesadores, adapters, editores o backends de materialización.

No son propietarios del contrato fuente.

La dirección de dependencia debe ser:

```
fuente IASI
    |
    v
interpretación IASI
    |
    +--> adapter Pandoc
    +--> adapter Quarto
    +--> integración con editores
    +--> backend de publicación
    +--> procesadores futuros
```

IASI conserva la propiedad semántica.

II.5.6. El front matter no basta

El front matter proporciona a IASI un lugar donde serializar metadatos.

No resuelve por sí mismo el problema semántico.

Por ejemplo, esto sería sintácticamente posible:

```
type = "inferred"
```

pero podría ser metodológicamente incorrecto si `inferred` es una propiedad de origen, mientras que `derived` pertenece a producción y `ephemeral` a ciclo de vida.

El esquema de metadatos debe reflejar las dimensiones conceptuales en lugar de forzarlas dentro de campos cómodos pero engañosos.

II.5.7. Capacidades futuras

El formato `.iasi` podrá incorporar en el futuro construcciones como:

- bloques ejecutables;
- referencias formales entre artefactos;
- afirmaciones embebidas;
- reglas de validación;
- identificadores semánticos;
- evidencia estructurada;
- enlaces de trazabilidad;
- instrucciones para Sistemas Inteligentes;
- directivas de materialización.

Estas capacidades deberían añadirse únicamente cuando preserven la separación central entre contenido y metadatos metodológicos.

II.5.8. Principio

`.iasi` es un contrato fuente de IASI, no un formato de otra herramienta con un nombre distinto.

Su sintaxis física debe mantenerse sencilla. Su semántica puede ser rica.

II.6. De la fuente a la materialización

IASI distingue la información fuente autoritativa de los artefactos producidos para consumo, publicación, ejecución o distribución.

Esta separación permite que los outputs generados sigan siendo útiles sin convertirse accidentalmente en fuentes de verdad.

II.6.1. El flujo

Un flujo simplificado es:

```
artefactos fuente
       |
       v
interpretación
       |
       v
transformación
       |
       v
materialización
       |
       v
artefactos derivados
```

El conjunto de fuentes puede incluir información `Authored`, externa e inferida que haya sido aceptada.

Los resultados materializados son normalmente derivados.

II.6.2. Fuente no significa `Authored`

Un artefacto fuente es simplemente una entrada de un proceso.

Puede ser:

- creado internamente;
- autoritativo externamente;
- inferido y posteriormente adoptado;
- otra fuente canónica.

IASI debe preservar estas distinciones mientras procesa el conjunto como entradas.

II.6.3. Outputs derivados

Entre los artefactos derivados materializados pueden encontrarse:

- sitios HTML;
- documentos PDF;
- presentaciones;
- diagramas generados;
- releases empaquetadas;
- esquemas generados;
- manifiestos de despliegue;
- árboles de publicación.

Un artefacto derivado puede conservarse por motivos de distribución o históricos, pero su existencia no debería elevarlo silenciosamente a autoridad independiente.

II.6.4. El principio del serrín

Los outputs de build y las áreas de staging son el **serrín** de la materialización.

Son consecuencias útiles del trabajo, no la fuente del trabajo.

Directorios como `_outputs` y `_publish` deben tratarse, por tanto, como regenerables y desechables.

Su contenido anterior no debe determinar la materialización siguiente.

Si la configuración actual solicita únicamente HTML, un PDF antiguo que permanezca en un directorio de trabajo no debe sobrevivir a la publicación por el mero hecho de existir de una ejecución previa.

Mandan la fuente actual y la selección actual.

II.6.5. Reproducción limpia

Un proceso de materialización sólido debe comportarse como si pudiera comenzar desde un estado derivado limpio.

Conceptualmente:

```
entradas autoritativas
        +
configuración actual
        +
reglas de transformación
        |
        v
output actual completo
```

No:

```
output antiguo
      +
algunos ficheros nuevos
      +
a cruzar los dedos
```

El segundo modelo permite que artefactos obsoletos se hagan pasar por resultados válidos.

II.6.6. Capas efímeras

Algunos artefactos derivados existen únicamente para soportar el proceso.

Por ejemplo:

```
fuentes
   |
   v
_outputs        <- derivado, regenerable
   |
   v
_publish        <- derivado, regenerable
   |
   v
release         <- artefacto derivado de publicación
```

Una release puede conservarse como registro durable, mientras que `_outputs` y `_publish` siguen siendo estructuras de trabajo descartables.

Durabilidad y derivación son propiedades independientes.

II.6.7. La materialización debe respetar los metadatos

Materializar no consiste únicamente en convertir ficheros.

El proceso puede depender de semántica metodológica.

Ejemplos:

- el contenido externo puede requerir atribución;
- el contenido inferido puede requerir enlaces a evidencias;
- los artefactos obsoletos pueden quedar excluidos;
- puede prohibirse utilizar artefactos derivados como entradas autoritativas;
- los artefactos confidenciales pueden tener destinos restringidos;
- los metadatos de versión pueden afectar a la publicación;
- las relaciones pueden determinar orden o inclusión.

Por eso los metadatos pertenecen al modelo IASI y no a un renderer concreto.

II.6.8. Principio

La fuente define la verdad; la materialización produce representaciones de esa verdad.

Los artefactos generados pueden ser importantes, distribuibles y durables. No deberían convertirse accidentalmente en la autoridad desde la que se reconstruye la metodología.

II.7. Inputs

Todo proyecto, proceso o actividad de ingeniería comienza con información.

Nada nace de la nada.

Antes de diseñar, decidir o construir existe algún tipo de información que provoca, condiciona o da sentido al trabajo. Puede ser un pliego, una presentación, una idea expresada en unas pocas líneas, una conversación, un correo, una especificación, un diagrama, una hoja de cálculo, código existente, una normativa, una incidencia, una URL o cualquier combinación de ellos.

Esa información no tiene por qué llegar preparada para ser procesada.

Puede aparecer en formatos diferentes, con distintos niveles de detalle y calidad. Puede estar estructurada o ser completamente informal; ser precisa o ambigua; estar completa o contener importantes lagunas; coincidir con otras fuentes o contradecirlas.

IASI denomina **inputs** al conjunto de información que entra en el proceso y que puede ser utilizada para comprender el problema, establecer sus límites y determinar qué acciones pueden realizarse.

`inputs/` constituye, por tanto, **la frontera de entrada del conocimiento en IASI**.

Su objetivo no es normalizar inmediatamente toda esa información ni obligarla a adoptar una forma común. Su función inicial es conservarla, clasificarla por su origen y ponerla a disposición del proceso manteniendo su significado y trazabilidad.

La transformación vendrá después.

```
información heterogénea
        ↓
      inputs/
        ↓
 análisis · contraste · reconciliación
        ↓
 comprensión y normalización
        ↓
   definitions/
```

La separación es fundamental:

> **`inputs/` conserva la información tal como entra, se aporta o se obtiene. `definitions/` representa esa información una vez comprendida y estructurada por IASI.**

Los inputs son la materia prima.

Las definitions son conocimiento ya formado para trabajar con él.

II.7.1. Nombre y ubicación

`inputs` es el nombre utilizado y recomendado por IASI.

Una implementación puede utilizar otro nombre físico cuando exista una razón para hacerlo. Lo importante no es la denominación concreta de la carpeta, sino conservar su semántica y su comportamiento.

La estructura de referencia distingue tres orígenes:

```
inputs/
   externals/
   internals/
   obtained/
```

La clasificación no depende del formato, del contenido ni de la importancia de la información.

Depende exclusivamente de **su procedencia**.

II.7.2. External inputs

`externals/` contiene la información recibida desde fuera del sistema de trabajo.

Por ejemplo:

- un pliego;
- una petición de un cliente;
- una presentación;
- una especificación recibida;
- documentación de un proveedor;
- una normativa;
- requisitos contractuales;
- un correo;
- documentación de un sistema de terceros;
- cualquier otro material proporcionado como entrada al proyecto.

Un input externo representa **lo que realmente se recibió**.

Por esa razón existe una regla estricta:

> **`inputs/externals/` es siempre de solo lectura para IASI.**

IASI puede leer, analizar, relacionar, contrastar e interpretar un input externo.

No puede modificarlo.

No debe:

- corregirlo;
- reescribirlo;
- normalizarlo;
- completarlo;
- eliminar información;
- cambiar su formato para adaptarlo al proceso;
- sustituir silenciosamente una versión por otra.

Si un pliego contiene un error, ese error forma parte del input recibido.

Si una presentación es ambigua, la ambigüedad debe conservarse.

Si dos documentos externos se contradicen, ambos deben permanecer intactos para que la contradicción pueda ser detectada, analizada y resuelta posteriormente.

> **Lo externo representa lo que recibimos, no lo que hubiéramos querido recibir.**

Cualquier corrección, interpretación o normalización debe producir **nueva información en otro lugar**.

Nunca debe alterar la evidencia original.

Si posteriormente llega una nueva versión de un documento externo, se incorpora como un nuevo input. La versión anterior se conserva mientras sea necesaria para mantener la trazabilidad del proceso.

II.7.3. Internal inputs

`internals/` contiene información aportada desde dentro del proceso de ingeniería.

Puede proceder del usuario, del equipo, de una decisión tomada durante el trabajo o de cualquier intervención interna que añada, restrinja o modifique el contexto disponible.

Por ejemplo:

```
Por ahora únicamente crea el proyecto.
```

Un input interno puede:

- añadir información;
- aclarar una petición anterior;
- establecer prioridades;
- introducir restricciones;
- delimitar el alcance;
- cambiar una decisión;
- indicar qué debe hacerse a continuación;
- indicar qué todavía no debe hacerse.

El nuevo input no modifica retroactivamente los anteriores.

Añade nueva información al proceso.

Esto permite conservar cómo evoluciona realmente el conocimiento y las decisiones durante un proyecto.

II.7.4. Obtained inputs

No toda la información necesaria es proporcionada explícitamente.

Parte de ella se descubre trabajando.

`obtained/` contiene información obtenida mediante actividades como:

- observación;
- análisis;
- inspección;
- exploración;
- ejecución;
- análisis de código;
- análisis de configuración;
- ingeniería inversa;
- pruebas;
- mediciones;
- consultas a sistemas existentes.

Por ejemplo, la documentación recibida puede no indicar qué versión de una biblioteca utiliza un sistema.

Si durante el análisis del repositorio IASI descubre esa versión, esa información no es externa y tampoco ha sido aportada mediante una decisión interna.

Es información **obtenida**.

Esta categoría permite distinguir con claridad:

```
lo que nos dijeron
lo que decidimos
lo que descubrimos
```

Esa diferencia puede ser esencial para la trazabilidad y para evaluar posteriormente la fiabilidad de una conclusión.

II.7.5. Diferentes formatos, una misma frontera

Los inputs no tienen que adoptar un formato documental único.

Un proyecto puede comenzar con:

```
inputs/
  externals/
      presentation.pdf
      requirements.xlsx
      architecture.png
      contract.pdf
      email.md
  internals/
      001-initial-scope.md
  obtained/
       001-existing-runtime.md
```

IASI debe aceptar que la realidad es heterogénea.

Forzar la normalización antes de incorporar la información produciría un problema importante: dejaríamos de conservar la entrada original y comenzaríamos a confundir **evidencia** con **interpretación**.

Por eso ambos niveles deben permanecer separados.

```
INPUT
"esto es lo que tenemos"
        ↓
ANÁLISIS
"esto es lo que entendemos"
        ↓
DEFINITION
"esto es lo que podemos utilizar"
```

II.7.6. Información activa e histórica

La información puede dejar de estar vigente sin dejar de ser relevante para la historia del proyecto.

Cada origen puede disponer de un espacio para información archivada:

```
inputs/
   externals/
       archived/
   internals/
       archived/
   obtained/
        archived/
```

Archivar un input no cambia su naturaleza.

Un external archivado continúa siendo external.

Un internal archivado continúa siendo internal.

Un obtained archivado continúa siendo obtained.

Archivar significa únicamente que esa entrada ya no participa en la interpretación del estado vigente.

Los procesos que calculen el contexto activo deben ignorar el contenido archivado.

Su conservación permite reconstruir posteriormente cómo evolucionó la información disponible.

II.7.7. Los inputs no son la solución

Recibir información no significa disponer de información suficiente para construir una solución.

Por ejemplo:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

Ese input expresa una intención.

Pero puede ser insuficiente para decidir:

- qué gráficos deben generarse;
- qué recibe el sistema como entrada;
- qué produce;
- cuál es la responsabilidad del filtro;
- cuál es la responsabilidad del MCP;
- qué tecnologías deben utilizarse;
- qué restricciones existen;
- cómo debe validarse el resultado.

IASI no debe completar silenciosamente esos huecos mediante suposiciones.

Debe ser capaz de distinguir entre:

```
lo conocido
lo inferido
lo desconocido
```

y determinar qué puede hacerse legítimamente con la información disponible.

II.7.8. Suficiencia y alcance

La información no necesita estar completa para que el trabajo pueda comenzar.

Necesita ser **suficiente para la acción concreta que se pretende realizar**.

Esta distinción es esencial.

El input:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

puede ser insuficiente para diseñar e implementar el sistema.

Sin embargo, posteriormente puede recibirse:

```
Por ahora únicamente crea el proyecto.
```

Este nuevo input no resuelve las incógnitas de la solución.

Lo que hace es **reducir explícitamente el alcance de la acción**.

IASI sigue sin saber lo suficiente para construir el sistema completo, pero sí puede disponer de información suficiente para crear el proyecto inicial.

Por tanto:

> **La falta de información limita el alcance; no necesariamente detiene el proceso.**

Y, de forma más precisa:

> **Si no sabemos lo suficiente para realizar una acción, no la realizamos. Podemos avanzar únicamente hasta donde la información disponible y los límites establecidos lo permitan.**

Esto permite que IASI avance de forma incremental sin inventar información y sin exigir tampoco una definición completa del proyecto antes de realizar cualquier trabajo.

II.7.9. Validación antes de actuar

Antes de ejecutar una acción, IASI debe evaluar el conjunto de inputs activos.

Entre otras cuestiones, debe determinar:

- qué información está disponible;
- cuál es su origen;
- qué información sigue vigente;
- si existen contradicciones;
- si existen ambigüedades relevantes;
- qué información falta;
- qué puede inferirse razonablemente;
- qué no puede afirmarse;
- si la información disponible es suficiente para la acción propuesta;
- hasta dónde permite avanzar el alcance establecido.

El objetivo no es conseguir que IASI siempre produzca una acción.

A veces el resultado correcto del análisis será:

```
no existe información suficiente para realizar esta acción
```

En otras ocasiones podrá concluir:

```
no puedo implementar la solución,
pero sí puedo realizar esta parte concreta
```

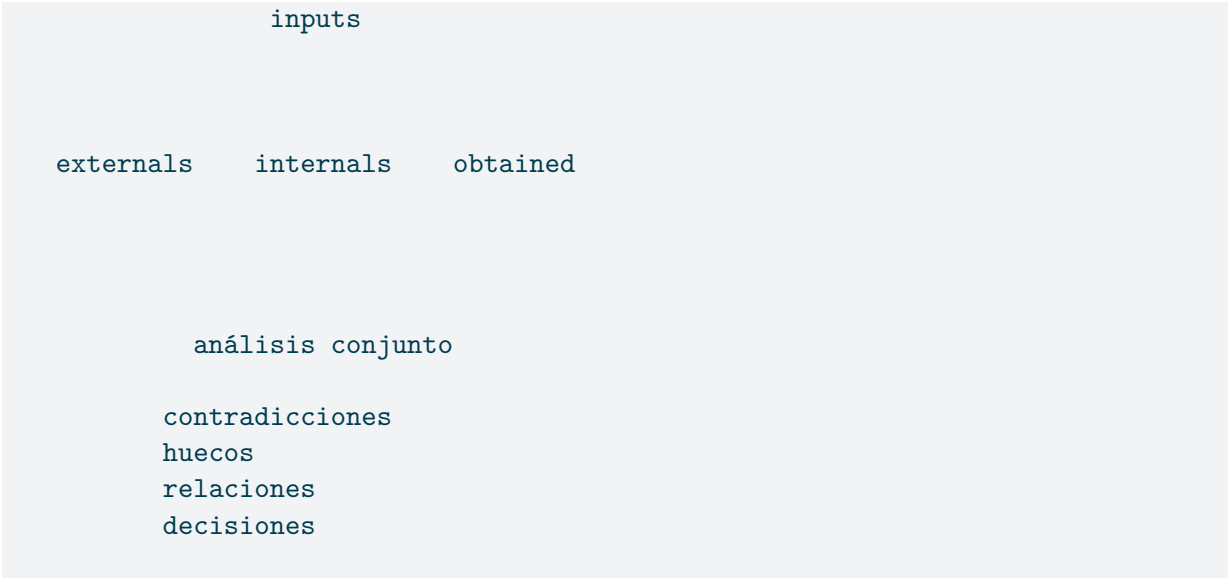
Ambos resultados representan comportamiento correcto.

II.7.10. De inputs a definitions

Los inputs no deben ser utilizados como sustituto permanente de una representación estructurada del proyecto.

IASI debe analizarlos, relacionarlos, contrastarlos y reconciliarlos.

De ese proceso nacen las `definitions`.



```
definitions
```

La relación no es necesariamente uno a uno.

Un único input puede contribuir a varias definitions.

Una definition puede construirse a partir de numerosos inputs.

Un input puede incluso no producir ninguna definition si resulta irrelevante, está superado o únicamente sirve como evidencia histórica.

IASI debe conservar siempre la diferencia entre ambos niveles:

> **Inputs responden a: "¿de dónde sale lo que sabemos?"**
>
> **Definitions responden a: "¿qué hemos entendido y estructurado para poder trabajar?"**

Mantener esa frontera permite que el conocimiento evolucione sin destruir sus fuentes y proporciona una base sólida para la trazabilidad, la validación y la reproducibilidad del proceso de ingeniería.

II.8. Definitions

Los inputs conservan información.

Las `definitions` representan **lo que IASI ha comprendido y estructurado a partir de esa información**.

La relación no es uno a uno.

Un único input puede contribuir a varias definitions y una definition puede necesitar múltiples inputs, decisiones y observaciones para quedar suficientemente formada.

```
inputs
     external
     internal
     obtained
         ↓
 análisis · contraste · reconciliación
         ↓
definitions
```

Una definition no debería ser una copia cosmetizada del documento de origen.

Su función es hacer explícito conocimiento utilizable por el resto del proceso: conceptos, restricciones, relaciones, límites, decisiones consolidadas, incógnitas relevantes o cualquier otra estructura que permita razonar con mayor precisión.

II.8.1. Conocido, inferido y desconocido

IASI debe conservar la diferencia entre:

```
conocido
inferido
desconocido
```

Una inferencia puede ser útil, pero no debe adquirir silenciosamente la misma autoridad que una evidencia o una decisión confirmada.

Del mismo modo, una incógnita relevante debe poder permanecer explícita. La metodología no exige rellenar todos los huecos antes de comenzar; exige no ocultarlos.

II.8.2. Generación y revisión

Las definitions pueden materializarse mediante un skill, un workflow, una herramienta o trabajo humano.

El mecanismo utilizado no cambia su responsabilidad.

El Sistema Inteligente puede proponer una interpretación. La condición canónica procede del proceso que la revisa, acepta y relaciona con sus fuentes, no de la confianza aparente con la que fue redactada.

II.8.3. Evolución

Cuando cambia un input activo puede ser necesario revisar las definitions que dependen de él. Ese cambio puede propagarse hacia specifications, tasks, validaciones o artefactos derivados. IASI no necesita regenerarlo todo indiscriminadamente. Necesita saber **qué queda afectado**.

II.9. Rules y configuración

Comprender el problema no basta para actuar.

El trabajo se realiza dentro de normas, convenciones, restricciones y parámetros que también deben hacerse explícitos.

IASI separa dos conceptos relacionados.

II.9.1. Rules

Una `rule` expresa una norma que debe, no debe o conviene respetar dentro de un ámbito.

Puede regular estilo, arquitectura, seguridad, documentación, compatibilidad, procesos o cualquier otro aspecto repetible de la ingeniería.

Las rules pueden ser generales o específicas. Cuando dos reglas regulan el mismo aspecto, **la más específica puede prevalecer sobre la genérica**.

Por ejemplo, una regla de estilo específica para Python puede refinar una regla de estilo genérica.

La plantilla normalizada de una rule puede contener:

```
título
metadata
rule
examples
exceptions
rationale
sources
```

Los ejemplos son opcionales, pero resultan útiles para mostrar casos positivos, negativos o límites de aplicación.

II.9.2. Configuration

La configuración expresa valores concretos que parametrizan el trabajo.

Una rule puede establecer que una tecnología crítica debe permanecer fijada durante un proyecto. La configuración indica **qué versión concreta** está fijada.

Una rule expresa política.

La configuration materializa parámetros.

Separar ambas permite cambiar valores sin reescribir la norma y reutilizar la misma norma en proyectos distintos.

II.10. Specifications

Una `specification` expresa **qué debe quedar satisfecho** por una unidad de trabajo o por un resultado.

No es una lista de ocurrencias del agente ni un registro de cómo decidió implementar algo.

Parte de:

```
definitions
+ rules
+ configuration
+ alcance
        ↓
specification
```

La specification transforma conocimiento y restricciones en una expectativa verificable.

Debe permitir responder, al menos, a estas preguntas:

- ¿qué resultado se espera?;
- ¿qué alcance tiene?;
- ¿qué restricciones son aplicables?;
- ¿qué queda explícitamente fuera?;
- ¿cómo sabremos que se ha satisfecho?

II.10.1. Specification y solución

Es importante no confundir una specification con la implementación.

La specification puede limitar tecnologías o imponer decisiones cuando existe una razón para ello, pero no debería describir innecesariamente cada paso de ejecución.

Su responsabilidad es establecer el contrato del resultado.

II.10.2. Criterios de aceptación

Siempre que sea posible, una specification debe conducir a criterios que permitan validar el resultado.

Una frase como «mejorar el rendimiento» es débil si no sabemos qué comportamiento debe cambiar ni cómo evaluar que la mejora es suficiente.

La metodología intenta que el salto entre intención y aceptación pueda recorrerse sin depender de interpretaciones retrospectivas.

II.11. Tasks

Las `tasks` convierten una specification en unidades de trabajo ejecutables.

Una task no crea por sí sola nuevo alcance.

Debe poder relacionarse con una necesidad, una specification o una decisión que justifique su existencia.

```
specification
     ↓
  tasks
     ↓
 ejecución
```

El tamaño adecuado de una task no se define por número de archivos o líneas de código.

Debe ser suficientemente concreta para poder:

- saber cuándo comienza;
- conocer qué pretende producir;
- identificar sus dependencias;
- ejecutarla de forma controlada;
- validar su resultado;
- determinar cuándo puede cerrarse.

II.11.1. Ejecución y estado

Una task atraviesa estados.

La capacidad para ejecutar una task no concede automáticamente permiso para cerrarla. La transición de estado debe respetar las validaciones y las reglas del workflow correspondiente.

IASI puede permitir que distintas personas, agentes o herramientas ejecuten tasks diferentes.

La unidad metodológica sigue siendo la task y su contrato, no la identidad de quien materializa el trabajo.

II.11.2. Resultado

La ejecución produce un `outcome`.

El outcome es el resultado observado de la operación. No debe confundirse con un argumento inventado para justificar retrospectivamente que una task está terminada.

Esta distinción será importante al estudiar los commands de task.

II.12. Validations

Una `validation` aporta evidencia sobre una condición que debe cumplirse.

IASI no considera que un artefacto sea correcto porque haya sido generado correctamente desde el punto de vista sintáctico ni porque un agente afirme haber terminado.

El resultado debe poder enfrentarse a criterios explícitos.

Las validations pueden comprobar, entre otras cosas:

- estructura;
- sintaxis;
- compilación;
- pruebas;
- reglas;
- consistencia entre artefactos;
- criterios de aceptación;
- presencia o ausencia de determinados efectos.

II.12.1. Validar no es generar

Separar generación y validación reduce un riesgo importante: que el mismo proceso que produjo una decisión se limite a declararla correcta.

No siempre será posible utilizar mecanismos completamente independientes, pero la metodología mantiene conceptualmente ambas responsabilidades.

```
ejecución
   ↓
resultado
   ↓
validation
   ↓
evidencia
```

II.12.2. Fallar también es un resultado

Una validation fallida no es un accidente que deba ocultarse.

Es información del proceso.

Puede bloquear transiciones posteriores, devolver trabajo a una fase anterior o exigir nueva información.

IASI debe conservar suficiente estado para que el fallo no desaparezca simplemente porque se inicia una nueva conversación o cambia el agente.

II.13. Estados y aceptación

Los estados permiten saber **dónde se encuentra el trabajo** sin reconstruir toda su historia cada vez.

Pero un estado no debe convertirse en una etiqueta decorativa.

Para que sea significativo debe estar respaldado por las condiciones que permiten alcanzarlo.

Una task puede estar iniciada, en ejecución, completada, fallida, reabierta o en cualquier otro estado definido por su ciclo de vida. Lo importante no es el vocabulario concreto, sino que las transiciones sean explícitas y reproducibles.

II.13.1. Done no significa necesariamente accepted

Completar el trabajo y aceptarlo son responsabilidades diferentes.

`done` puede expresar que la ejecución prevista ha finalizado.

La aceptación expresa que el resultado satisface las condiciones que justifican su incorporación al estado válido del proyecto.

```
trabajo realizado
      ↓
validaciones
      ↓
evidencia
      ↓
aceptación
      ↓
estado consolidado
```

Esta separación permite que IASI conserve una frontera clara entre **hacer** y **dar por bueno**.

II.13.2. El estado es materializado

El estado relevante no debe depender de que una persona o un agente recuerden qué ocurrió.

Debe quedar persistido de forma que el siguiente participante pueda continuar el trabajo con el mismo punto de partida observable.

II.14. Cambio e iteración

Los proyectos cambian.

Llegan nuevos documentos, se toman decisiones, aparecen restricciones, se descubren datos y se invalidan supuestos anteriores.

IASI no intenta eliminar ese cambio. Intenta **hacer visible su impacto**.

Cuando cambia una fuente, la pregunta no es solamente «¿qué fichero debo regenerar?».

La pregunta es:

> **¿qué conocimiento, decisiones y resultados dependen de aquello que ha cambiado?**

```
nuevo input
   ↓
definitions afectadas
   ↓
specifications afectadas
   ↓
tasks / validations afectadas
   ↓
artefactos derivados
```

II.14.1. Invalidación

Una validación puede ser correcta para un conjunto concreto de entradas y dejar de ser válida cuando esas entradas cambian.

Por eso determinados estados pueden asociarse a hashes o identificadores de las fuentes que justificaron su resultado.

Cambiar la dependencia invalida la conclusión anterior.

II.14.2. Archivar no es borrar

Un input que deja de estar activo puede archivarse sin perder su naturaleza ni su historia.

El material archivado deja de participar en el contexto vigente, pero sigue permitiendo reconstruir cómo evolucionó el proyecto.

II.14.3. Contenido parcialmente derivable

Algunos artefactos combinan partes derivables y contenido editorial o profesional añadido manualmente.

IASI debe regenerar únicamente aquello que pueda derivarse de forma segura.

Si cambia una dependencia de contenido mantenido por autor, el comportamiento correcto puede ser marcarlo como **affected** y solicitar revisión, no reescribirlo silenciosamente.

Parte III.

Estructura de un proyecto IASI

La metodología define responsabilidades.

La estructura del proyecto les proporciona un lugar estable.

IASI utiliza directorios y artefactos con significado conocido para que personas, Sistemas Inteligentes y herramientas puedan encontrar información sin depender de convenciones implícitas de cada equipo.

La estructura es un **contrato operativo**, no una decoración del repositorio.

No todos los proyectos necesitan materializar todos los directorios desde el primer minuto. La estructura puede crecer conforme aparecen capacidades y artefactos. Lo importante es que, cuando una categoría existe, conserve su semántica.

III.1. La raíz del proyecto

Una estructura de referencia puede tomar esta forma:

```
project/
   .iasi/
   inputs/
   definitions/
   rules/
   configuration/
   specifications/
   tasks/
   validations/
   states/
   outputs/
   releases/
   .codex/
   .copilot/
```

La presencia concreta depende de las capacidades utilizadas.

IASI no exige crear carpetas vacías para simular completitud.

III.1.1. Dos familias de directorios

Conviene distinguir:

Artefactos de ingeniería

```
inputs/
definitions/
rules/
configuration/
specifications/
tasks/
validations/
states/
outputs/
releases/
```

representan información y resultados del proyecto.

Infraestructura de IASI y de los entornos

```
.iasi/
.codex/
.copilot/
...
```

permite que la metodología se materialice y sea utilizada por herramientas concretas.

La diferencia es importante porque las carpetas específicas de un agente no deben convertirse en la fuente de verdad del proyecto.

III.1.2. Estructura semántica

Cambiar un nombre físico puede ser posible cuando exista una razón y la configuración lo soporte.

Pero hacerlo no debe borrar la semántica.

Un directorio llamado de otra manera sigue necesitando una responsabilidad inequívoca. IASI no debería tener que adivinar si una carpeta contiene evidencia original, conocimiento normalizado o resultados derivados.

III.1.3. Estructura mínima y estructura creciente

La estructura de referencia no significa que todo proyecto deba comenzar con todas las carpetas vacías.

IASI distingue entre **semántica requerida** y **materialización necesaria en cada momento**.

Un proyecto crea los espacios que necesita conforme utiliza las capacidades correspondientes. Lo importante es que, cuando exista un artefacto, su ubicación y su significado sean inequívocos.

Por tanto, la estructura puede crecer con el proyecto sin perder el contrato común.

III.2. Estructura de inputs

La estructura de referencia de los inputs distingue su procedencia:

```
inputs/
   externals/
       archived/
   internals/
       archived/
   obtained/
        archived/
```

La clasificación responde a tres preguntas diferentes:

```
externals → ¿qué recibimos?
internals → ¿qué aportamos o decidimos?
obtained  → ¿qué descubrimos trabajando?
```

Los subárboles `archived/` conservan información histórica que ya no participa en el contexto activo.

El archivado no cambia la naturaleza del input.

Un external archivado continúa siendo external. Simplemente deja de intervenir en el cálculo vigente.

III.2.1. Solo lectura de externals

Desde el punto de vista de IASI:

> **`inputs/externals/` es de solo lectura.**

Cualquier corrección, normalización o interpretación produce un nuevo artefacto fuera de ese directorio.

Esta propiedad permite utilizar los inputs externos como evidencia estable incluso cuando el conocimiento derivado evoluciona.

III.3. Artefactos de ingeniería

Después de la frontera de inputs aparecen los directorios que materializan el proceso de ingeniería:

```
definitions/
rules/
configuration/
specifications/
tasks/
validations/
states/
```

No forman una colección arbitraria.

Cada uno responde a una pregunta distinta:

Directorio	Pregunta
`definitions/`	¿qué hemos comprendido?
`rules/`	¿qué normas debemos respetar?
`configuration/`	¿con qué parámetros trabajamos?
`specifications/`	¿qué debe quedar satisfecho?
`tasks/`	¿qué trabajo concreto ejecutamos?
`validations/`	¿qué evidencia comprueba el resultado?
`states/`	¿cuál es el estado materializado?

La metodología puede añadir especialización interna dentro de cada uno.

Por ejemplo, las rules pueden organizarse por dominio, tecnología o lenguaje. Esa organización no cambia su naturaleza normativa.

III.3.1. No confundir estructura con secuencia

Que estos directorios puedan representarse en un flujo no significa que únicamente puedan escribirse en ese orden.

Una validación puede descubrir información que produzca un nuevo input obtenido. Una modificación de configuration puede afectar specifications existentes. Un cambio de rule puede invalidar trabajo previamente aceptado.

La estructura conserva responsabilidades.

El workflow decide cómo se recorren.

III.4. Estado local de IASI

`.iasi/` contiene información de control necesaria para que IASI opere dentro de un proyecto.

Parte de esa información puede componerse o heredarse entre ámbitos. Otra debe ser estrictamente local.

Un ejemplo importante es:

```
.iasi/validation.json
```

El resultado persistente de validación pertenece al proyecto o ámbito que fue validado.

No se hereda.

Heredar un estado de validación produciría una conclusión falsa: que un hijo está validado porque lo estaba su padre aunque sus entradas, reglas o contenido sean distintos.

III.4.1. Composición entre ámbitos

Cuando existen configuraciones IASI en distintos niveles, pueden componerse de padre a hijo.

La resolución se realiza desde el ámbito más general hacia el más próximo, y el más próximo puede refinar aquello que resulte heredable.

Este mecanismo permite compartir comportamiento sin duplicarlo en todos los proyectos.

Pero la herencia no debe aplicarse indiscriminadamente.

Cada artefacto debe declarar o tener definida su semántica de composición.

III.5. Directorios derivados de adapters

Un proyecto puede utilizar uno o varios agentes o entornos.

Por ejemplo:

```
.iasi/
.codex/
.copilot/
```

La relación entre ellos no es simétrica.

> **`.iasi/` representa la información canónica de IASI; los directorios específicos de herramientas son proyecciones derivadas.**

Un adapter puede materializar prompts, instrucciones, configuraciones, skills u otros archivos que un entorno concreto necesite.

Esos archivos existen para integrar IASI con la herramienta.

No deben convertirse en el lugar donde se redefine la metodología.

```
IASI canónico
     ↓
   adapter
     ↓
.codex/ · .copilot/ · ...
```

Si cambia el formato requerido por una herramienta, cambia el adapter.

No debería ser necesario reescribir el command o workflow canónico para seguir utilizándolo.

III.6. Outputs y releases

`outputs/` y `releases/` representan momentos diferentes del ciclo de vida de un resultado.

III.6.1. Outputs

`outputs/` contiene resultados materializados por el trabajo.

Pueden ser código, documentos, paquetes, informes, binarios u otros artefactos derivados.

Que algo aparezca en `outputs/` no significa necesariamente que esté aceptado o publicado.

III.6.2. Releases

`releases/` contiene resultados preparados para entrega o publicación según las reglas del proyecto.

```
trabajo
   ↓
outputs/
   ↓
validación y aceptación
   ↓
releases/
```

III.6.3. Evitar realimentación accidental

Las herramientas utilizadas para construir un resultado pueden producir sus propios directorios temporales.

Esos artefactos intermedios no deberían mezclarse con la salida final si eso hace que vuelvan a entrar como fuente del siguiente build.

Cuando una herramienta necesita un directorio de construcción, conviene aislarlo y enrutar únicamente el resultado final hacia `outputs/`.

La regla general es sencilla:

> **Los resultados derivados no deben reaparecer accidentalmente como nuevas entradas del mismo proceso de materialización.**

Parte IV.

Workflows

La metodología define las responsabilidades y los artefactos.

Un `workflow` define **cómo recorremos esas responsabilidades para alcanzar un objetivo concreto**.

Dos workflows pueden utilizar los mismos inputs, rules, commands y validations en órdenes o combinaciones diferentes sin que la metodología cambie.

```
metodología
   ↓
workflows
   ↓
ejecuciones concretas
```

Un workflow puede incluir trabajo humano, llamadas a un Sistema Inteligente, herramientas deterministas, validaciones automáticas y decisiones que requieran autoridad profesional.

IV.1. Modelo de workflow

Un workflow es una secuencia coordinada de actividades con condiciones de entrada, transiciones y criterios de salida.

No es simplemente una lista de pasos escrita en un prompt.

Un workflow debería poder hacer explícitos:

- objetivo;
- entradas requeridas;
- artefactos que puede leer;
- artefactos que puede modificar;
- commands que utiliza;
- validaciones necesarias;
- estados que puede producir;
- puntos de decisión;
- condiciones de finalización.

La misma definición puede ejecutarse de formas diferentes.

Una persona puede realizar manualmente determinados pasos. Un agente puede ejecutar otros. Una automatización puede encadenarlos cuando existan condiciones suficientes.

La semántica del workflow no depende de quién pulsa el botón.

IV.1.1. Workflow y metodología

La metodología define las reglas del juego.

El workflow define una forma válida de jugar una partida.

Por eso un workflow no puede saltarse una validación obligatoria simplemente porque el agente utilizado tenga capacidad técnica para continuar.

IV.2. De inputs a definitions

Uno de los workflows fundamentales transforma información de entrada en conocimiento utilizable.

```
inputs activos
     ↓
validar disponibilidad y origen
     ↓
detectar contradicciones y huecos
     ↓
analizar y relacionar
     ↓
proponer definitions
     ↓
revisar / aceptar
```

El objetivo no es resumir todos los documentos.

Es producir definitions que permitan continuar el trabajo sin perder la relación con sus fuentes.

El workflow debe poder terminar también sin crear una definition.

Eso puede ocurrir cuando el input es irrelevante, está superado, contiene información insuficiente o únicamente aporta evidencia histórica.

La ausencia de un artefacto nuevo puede ser un resultado correcto.

IV.3. Planificación y ejecución

Una vez existe conocimiento suficiente, IASI puede convertir una necesidad en trabajo ejecutable.

```
definitions
+ rules
+ configuration
      ↓
specifications
      ↓
tasks
      ↓
ejecución
```

La planificación no necesita descomponer por adelantado todo el proyecto hasta el último detalle.

Puede materializar únicamente aquello que el alcance actual permite decidir con suficiente confianza.

Las tasks se ejecutan dentro del contrato de su specification y respetando las rules aplicables.

El ejecutor puede ser una persona, un Sistema Inteligente, una herramienta o una combinación de ellos.

Lo importante es que el resultado vuelva al proceso como un artefacto observable y pueda ser validado.

IV.4. Validación y aceptación

El workflow de validación comienza cuando existe un resultado que debe contrastarse con una condición.

```
resultado
   ↓
validations aplicables
   ↓
evidencia
     fail → revisar / corregir
     pass → candidato a aceptación
                  ↓
               aceptar
                  ↓
                state
```

El paso de `pass` a aceptación no debe confundirse con una operación puramente mecánica.

Hay contextos en los que la aceptación puede automatizarse porque los criterios son completamente objetivos y la autoridad ha sido delegada de forma explícita.

En otros, requiere decisión humana.

IASI debe permitir ambos casos sin ocultar quién o qué tenía autoridad para efectuar la transición.

IV.5. Workflow de cambio

Un cambio comienza con nueva información, no con la obligación de regenerar todo el proyecto.

```
nuevo input
   ↓
clasificar origen
   ↓
incorporar sin destruir historia
   ↓
analizar impacto
   ↓
marcar affected / invalidar
   ↓
actualizar únicamente lo necesario
   ↓
volver a validar
```

La propagación depende de las relaciones de trazabilidad.

Si una definition no depende del input modificado, no existe razón metodológica para reconstruirla.

Si un artefacto es parcialmente derivable, puede regenerarse su parte automática y conservar la parte de autor.

El workflow de cambio convierte la trazabilidad en una herramienta operativa: permite avanzar sin tratar cada modificación como si el proyecto comenzara de cero.

Parte V.

Commands

Los commands proporcionan un vocabulario canónico para solicitar operaciones IASI.

Un command no pertenece a Codex, Copilot, Claude ni a ningún otro agente.

Su significado pertenece a IASI.

El adapter decide cómo presentar o invocar esa operación en cada entorno.

```
command IASI
     ↓
   adapter
     ↓
sintaxis / mecanismo del entorno
```

Esta separación permite que el mismo workflow sobreviva al cambio de herramienta.

V.1. Modelo de command

Un command debe poder describirse sin depender de la interfaz que lo invoque.

Conceptualmente contiene:

```
nombre
intención
parámetro principal
parámetros adicionales
precondiciones
efectos
resultado
transiciones permitidas
```

V.1.1. Parámetro principal

Cuando un command posee un parámetro natural, puede declararlo como parámetro principal.

Esto permite utilizar binding posicional o nombrado sin introducir ambigüedad.

V.1.2. Literales y referencias

IASI distingue conceptualmente entre un literal y una referencia.

Una cadena entre comillas representa un valor literal.

Un identificador sin comillas puede representar una referencia resoluble por el command.

V.1.3. Selectors

Los selectors permiten aplicar una operación a cero, uno o varios artefactos.

Un selector que no encuentra coincidencias no debe transformarse silenciosamente en otra operación. El resultado puede ser un warning y ninguna modificación.

V.1.4. Outcome

El outcome describe lo que ocurrió al ejecutar la operación.

No es un argumento que el llamador pueda inventar para imponer el estado final.

V.2. validate

`/validate` fue el primer command canónico definido para IASI.

Su responsabilidad es comprobar que las condiciones necesarias permiten continuar el proceso.

En su forma inicial evalúa:

```
instructions
+
inputs activos
```

y materializa un resultado persistente:

```
.iasi/validation.json
```

V.2.1. Fallo persistente

Si la validación falla, el estado no desaparece al terminar la conversación.

El resultado persistente bloquea las fases posteriores que dependan de esa validación.

Esto evita que un nuevo agente o una nueva sesión continúen simplemente porque no conocen el fallo anterior.

V.2.2. Invalidación por cambio

Una validación solo es válida respecto al conjunto de entradas que fue comprobado.

IASI puede utilizar hashes de esas dependencias.

Si cambian, la validación anterior deja de ser aplicable y debe ejecutarse de nuevo.

V.2.3. Estado local

`.iasi/validation.json` es local.

No debe heredarse desde un proyecto padre, porque la validez de un ámbito no demuestra la validez de otro.

V.3. Commands de task

IASI dispone de un pequeño vocabulario para controlar el ciclo de vida de una task.

La forma canónica acordada es:

```
task.start(selector)
task.run(selector, completion = "auto")
task.done(selector)
task.rerun(selector, completion = "auto")
task.reopen(selector)
```

V.3.1. `task.start`

Inicia una task seleccionada cuando sus precondiciones permiten comenzar.

V.3.2. `task.run`

Ejecuta la task.

`completion = "auto"` permite que la operación complete automáticamente la transición prevista cuando las condiciones del workflow lo permitan.

V.3.3. `task.done`

Materializa la finalización de una task.

El outcome procede de la ejecución y de la evidencia observada. No se introduce como argumento de `task.done` para forzar una conclusión.

V.3.4. `task.rerun`

Permite volver a ejecutar trabajo desde estados como `done` o `failed` cuando el workflow lo autoriza.

V.3.5. `task.reopen`

Reabre trabajo que ya había sido cerrado para permitir una nueva intervención explícita.

La existencia de commands diferentes hace visibles transiciones que de otro modo quedarían ocultas dentro del comportamiento del agente.

V.4. archive

Archivar un input significa retirarlo del contexto activo sin destruir su historia.

La operación mantiene la rama lógica del documento:

```
inputs/externals/... → inputs/externals/archived/...
inputs/internals/... → inputs/internals/archived/...
inputs/obtained/...  → inputs/obtained/archived/...
```

El nombre archivado puede incorporar una marca temporal `YYYYMMDDhhmmss` para conservar versiones sin colisiones.

V.4.1. Efecto sobre la validación

Los subárboles `archived/` se excluyen del conjunto de inputs activos y, por tanto, de los hashes utilizados para validar el estado vigente.

Archivar no reescribe el documento.

No cambia su procedencia.

Solo modifica su participación en el contexto activo.

V.5. adapter

El command canónico para materializar la integración con un agente adopta la forma:

```
adapter(agent, dir=".<agent>", overwrite=false)
```

Por ejemplo:

```
adapter(agent="codex", dir=".codex", overwrite=false)
```

El command selecciona un adapter conocido y proyecta sus artefactos sobre el directorio de destino.

V.5.1. `overwrite`

Por defecto, la materialización no debe destruir silenciosamente contenido existente.

`overwrite=false` obliga a tratar una colisión como una condición explícita.

V.5.2. Responsabilidad

`adapter` no cambia la metodología.

Materializa una representación compatible con un entorno concreto a partir de información canónica de IASI.

Por eso el resultado del adapter es **derivado** y puede regenerarse.

Parte VI.

Adapters e integración

IASI pretende vivir sobre cualquier agente y, cuando sea necesario, sobre distintos modelos o runtimes.

Eso solo es posible si la metodología no adopta como propios los formatos particulares de una herramienta.

Los adapters forman esa frontera.

> **IASI no se adapta metodológicamente al agente. El adapter adapta el agente a IASI.**

Esta parte desarrolla cómo se conserva una semántica canónica mientras cambian los entornos de ejecución.

VI.1. Por qué existen los adapters

Cada agente posee convenciones diferentes.

Puede buscar instrucciones en directorios distintos, utilizar formatos propios para commands o skills, ofrecer herramientas diferentes y tener mecanismos específicos para cargar contexto.

Si la metodología dependiera directamente de esas convenciones, terminaríamos manteniendo varias IASI:

```
IASI para Codex
IASI para Copilot
IASI para Claude
IASI para ...
```

El adapter evita esa fragmentación.

```
semántica IASI
      ↓
   adapter
      ↓
semántica del host
```

El núcleo define la operación.

El adapter conoce el idioma operativo del entorno.

VI.2. Canónico y derivado

La distinción principal es:

```
.iasi/      → canónico dentro del proyecto
.codex/     → proyección derivada
.copilot/   → proyección derivada
...
```

Los nombres concretos de los directorios derivados pueden configurarse, pero su naturaleza no cambia.

Un archivo generado para un agente no debe adquirir autoridad simplemente porque ese agente lo consume directamente.

Si existe una discrepancia entre la representación derivada y la fuente canónica, debe corregirse la fuente o el adapter y volver a materializarse.

Modificar únicamente la proyección crea una bifurcación invisible.

VI.2.1. Regenerabilidad

Una buena proyección debe poder reconstruirse.

Eso convierte los directorios de adapter en resultados controlados, no en nuevos centros de conocimiento.

VI.3. Adapters de agente

Un adapter de agente traduce capacidades IASI a los mecanismos que un entorno concreto ofrece.

Puede materializar:

- instrucciones;
- commands;
- skills;
- configuración;
- referencias a tools;
- convenciones de directorio;
- otros artefactos requeridos por el host.

La traducción debe ser lo más delgada posible.

El adapter no debería contener una segunda implementación independiente de la metodología.

Su trabajo consiste en proyectar la definición canónica y resolver diferencias de formato o integración.

VI.3.1. Adaptación atómica

La materialización de un adapter debe tratarse como una operación controlada.

Si no puede producir una proyección coherente, es preferible fallar de forma explícita que dejar un entorno a medio adaptar y permitir que el agente trabaje con una mezcla de versiones.

VI.4. Agente, modelo y runtime

Agente y modelo no son la misma dependencia.

Un agente puede utilizar diferentes modelos.

Un mismo modelo puede aparecer detrás de agentes o runtimes distintos.

IASI evita acoplar ambas decisiones.

```
IASI
 ↓
adapter de agente / entorno
 ↓
runtime
 ↓
modelo o proveedor
```

En algunos casos el adapter necesita conocer capacidades del runtime o del modelo disponible.

Eso no convierte esas capacidades en requisitos universales de la metodología.

La separación permite sustituir un proveedor de modelo sin tener que redefinir el significado de una task, una validation o un command.

VI.5. Capacidades y degradación

No todos los entornos pueden hacer lo mismo.

Un agente puede disponer de edición de archivos, ejecución de shell, acceso a red, MCP, memoria persistente o ninguna de esas capacidades.

El adapter debe conocer las capacidades relevantes antes de prometer una operación.

VI.5.1. No simular garantías

Cuando falta una capacidad necesaria, IASI no debería fingir que la garantía se mantiene.

Las respuestas posibles dependen del workflow:

- utilizar un mecanismo alternativo equivalente;
- degradar explícitamente la operación;
- solicitar intervención humana;
- bloquear la transición.

Lo importante es que la diferencia sea visible.

> **La portabilidad no consiste en fingir que todos los agentes son iguales. Consiste en conservar la semántica de IASI a pesar de que no lo sean.**

Parte VII.

Mecanismos transversales

Algunos elementos de IASI no pertenecen a una única fase del ciclo.

Rules, skills, templates, configuration y automations pueden intervenir en numerosos workflows y afectar a múltiples clases de artefactos.

Esta parte los estudia como mecanismos reutilizables.

También recoge dos problemas transversales: cómo resolver alcance y precedencia y cómo combinar contenido derivado con aportaciones mantenidas por personas.

VII.1. Rules

Una rule materializa una norma reutilizable.

Su estructura normalizada puede expresarse en Markdown mediante:

```
# <Rule name>

## Metadata
## Rule
## Examples
## Exceptions
## Rationale
## Sources
```

`Examples` es opcional. Puede contener casos positivos, negativos o situaciones límite cuando ayuden a comprender la norma.

VII.1.1. Nivel normativo

La metadata puede distinguir niveles como:

```
must
must-not
should
should-not
may
```

Esto evita expresar con el mismo peso una obligación y una recomendación.

VII.1.2. Especificidad

Cuando una rule específica y otra genérica regulan el mismo aspecto, IASI puede aplicar la más específica.

Por ejemplo:

```
coding/styles/generic/*
coding/styles/python/*
```

Para código Python, la regla específica del lenguaje prevalece cuando ambas entran en conflicto sobre el mismo asunto.

La especificidad permite reutilizar principios generales sin impedir que un dominio introduzca sus propias restricciones.

VII.2. Skills

Un skill encapsula conocimiento operativo reutilizable para realizar una clase de trabajo.

No es una fuente de verdad del proyecto.

Es un mecanismo que ayuda a transformar artefactos respetando la metodología.

Por ejemplo, la generación de definitions a partir de inputs puede apoyarse en un skill que indique cómo:

- identificar fuentes;
- extraer conceptos;
- detectar contradicciones;
- conservar incertidumbre;
- relacionar cada conclusión con evidencia;
- producir la estructura canónica esperada.

El skill puede ser utilizado por distintos agentes.

Su valor reside precisamente en que la forma de trabajar no queda encerrada en el prompt improvisado de una sesión.

VII.2.1. Skill y command

Un command expresa una operación.

Un skill aporta conocimiento para ejecutarla.

Pueden colaborar, pero no son equivalentes.

VII.3. Templates

Los templates proporcionan formas canónicas para artefactos repetibles.

Una plantilla no debería obligar a rellenar contenido inexistente.

Su función es estabilizar estructura, metadata y convenciones allí donde esa estabilidad aporta valor.

Por ejemplo, una plantilla de rule puede garantizar que todas las rules expresen de la misma forma su ámbito, nivel normativo, excepciones y fuentes.

VII.3.1. Plantilla y semántica

La plantilla determina la forma esperada.

No decide el contenido.

Un Sistema Inteligente puede utilizarla para materializar un nuevo artefacto, pero sigue necesitando inputs y autoridad suficientes para completar cada sección.

VII.3.2. Templates incorporados

IASI puede distribuir templates de fábrica dentro de su contenido `builtin/`.

Un proyecto puede reutilizarlos o especializarlos sin convertir una copia modificada en una excepción oculta del runtime.

VII.4. Configuration

La configuration reúne parámetros que modifican el comportamiento del proceso sin redefinir su metodología.

Puede fijar:

- versiones;
- rutas;
- nombres físicos;
- opciones de herramientas;
- valores por defecto;
- capacidades habilitadas;
- otras decisiones parametrizables.

Una configuración importante debe ser persistente y revisable.

VII.4.1. Estabilidad tecnológica

Durante un proyecto, las tecnologías críticas deberían mantenerse fijadas salvo que exista una necesidad justificada para cambiarlas.

Que aparezca una versión más reciente no convierte automáticamente la actualización en una buena decisión.

Un cambio de versión puede afectar compatibilidad, arquitectura, dependencias, pruebas, despliegue o certificaciones.

La configuration permite que esa decisión sea explícita en lugar de depender del entorno accidental de quien ejecuta el trabajo.

VII.5. Automations

Una automation ejecuta de forma automática una operación o parte de un workflow cuando se cumplen determinadas condiciones.

No debe confundirse con el workflow.

El workflow define la lógica del recorrido.

La automation elimina intervención manual en uno o varios puntos de ese recorrido.

```
workflow
     actividad humana
     command
     automation
     validation
```

Automatizar no cambia la autoridad de una transición.

Si una aceptación requiere decisión humana, envolver el proceso en una automation no elimina ese requisito.

La automatización es válida cuando conserva las precondiciones, validaciones y límites establecidos por la metodología.

VII.6. Ámbito y precedencia

IASI necesita combinar información procedente de distintos niveles.

Puede existir configuración común para varios proyectos, reglas compartidas y especializaciones locales.

La resolución debe ser explícita.

VII.6.1. De lo general a lo próximo

En estructuras componibles, IASI puede construir el contexto desde el ámbito padre hacia el hijo.

El ámbito más próximo puede refinar aquello que sea heredable.

```
general
  ↓
organización
  ↓
proyecto
  ↓
subproyecto
```

No todo se hereda.

El estado local de validación es un ejemplo claro de información que debe permanecer vinculada al ámbito concreto.

VII.6.2. Especificidad

La proximidad no es la única dimensión.

Las rules pueden resolver también por especificidad semántica: una regla de lenguaje es más específica que una regla genérica de coding para ese lenguaje.

VII.6.3. Resolver, no adivinar

Cuando no existe una regla de precedencia aplicable y dos fuentes incompatibles reclaman autoridad sobre el mismo aspecto, el resultado correcto es un conflicto explícito.

IASI no debería esconderlo escogiendo una opción al azar.

VII.7. Contenido derivado y contenido de autor

No todos los artefactos son completamente generables ni completamente manuales.

Una guía de usuario, por ejemplo, puede derivar gran parte de su estructura y contenido del conocimiento del proyecto y contener al mismo tiempo explicaciones, ejemplos o decisiones editoriales mantenidas por una persona.

IASI debe permitir esa combinación sin convertir la regeneración en una trituradora de trabajo humano.

Una estrategia útil separa:

```
contenido derivable
+
includes de autor
```

Al regenerar:

- IASI actualiza lo derivable;
- conserva los includes de autor;
- detecta si las dependencias del contenido de autor han cambiado;
- marca ese contenido como `affected` cuando necesita revisión;
- no lo reescribe automáticamente salvo que exista autorización explícita.

Este patrón generaliza más allá de la documentación.

Permite trabajar con cualquier artefacto parcialmente derivable manteniendo una frontera clara entre **materialización automática** y **aportación profesional deliberada**.

Parte VIII.

Saco

Documentos pendientes de integrar

Referencias

Las referencias bibliográficas utilizadas en este volumen se incorporarán a medida que los capítulos requieran fuentes externas. Los elementos normativos propios de IASI se documentan, cuando procede, junto con sus fuentes y su justificación.

Licencias

El proyecto **iasi** contiene materiales de distinta naturaleza. Cada tipo de material se distribuye bajo una licencia específica.

Copyright © 2026 Javier G. Grandez.

Código fuente

El código fuente, los scripts, las automatizaciones, las utilidades y los ejemplos ejecutables se distribuyen bajo la **Licencia MIT**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- scripts escritos en R;
- filtros, extensiones y utilidades;
- archivos de automatización;
- herramientas de construcción y publicación;
- fragmentos y ejemplos de código reutilizables.

La Licencia MIT permite utilizar, copiar, modificar, integrar, publicar, distribuir, sublicenciar y vender el código, siempre que se conserve el aviso de copyright y el texto de la licencia.

El texto completo está disponible en:

Licencia MIT

Identificador SPDX: `MIT`.

Contenido editorial

Los textos, capítulos, diagramas, tablas e imágenes originales de la obra **Ingeniería aumentada con sistemas inteligentes** se distribuyen bajo la licencia **Creative Commons Attribution 4.0 International**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- los documentos y capítulos escritos en formato `.qmd`;
- los diagramas, tablas e ilustraciones originales;
- las versiones HTML, PDF y otros formatos generados a partir de la obra.

Esta licencia permite compartir y adaptar el contenido, incluso con fines comerciales, siempre que se reconozca adecuadamente la autoría, se proporcione una referencia a la licencia y se indiquen los cambios realizados.

El texto legal completo está disponible en:

Creative Commons Attribution 4.0 International

Identificador SPDX: `CC-BY-4.0`.

Atribución

Para reutilizar el contenido editorial, se recomienda utilizar una atribución semejante a la siguiente:

> Javier G. Grandez, *Ingeniería aumentada con sistemas inteligentes*, 2026. Licencia CC BY 4.0.

Cuando el contenido haya sido modificado, deberá indicarse de forma razonable que se han realizado cambios.

Materiales de terceros

Los materiales pertenecientes a terceros conservan sus respectivas licencias, derechos de autor y condiciones de uso.

Su inclusión o referencia en este proyecto no implica que estén cubiertos por la Licencia MIT ni por la licencia CC BY 4.0.

Cuando corresponda, la autoría, la procedencia y la licencia aplicable se indicarán junto al material o en su referencia bibliográfica.

Alcance

La Licencia MIT se aplica al **código**.

La licencia CC BY 4.0 se aplica al **contenido editorial original**.

Cuando un archivo incluya una indicación de licencia específica, esa indicación prevalecerá para dicho archivo.